ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 6.

Դասերի օգտագործմամբ բարդ խնդիրներ (C++)

Աշխատանքի նպատակ. Ծանոթանալ C++ ծրագրավորման լեզվում դասերի (`class`), օբյեկտների, կոնստրուկտորների և մեթոդների օգտագործմանը: Զարգացնել տվյալների մուտքագրման, ստուգման, որոնման և փոփոխման հմտությունները՝ կիրառելով դասերի մեթոդներ, պայմաններ և ցիկլեր:

Խնդրի պահանջ. Ստեղծել `Patient` դաս՝ `id`, `name`, `severity`, `arrivalMinute`, `served` դաշտերով: Դաշտերը պետք է լինեն `private`, իսկ տվյալների հետ աշխատելու համար պետք է սահմանվեն `public` կոնստրուկտոր և մեթոդներ: Ստեղծել 15 հիվանդի գրառում:

Ծրագիրը պետք է ընտրի սպասարկման հաջորդ հիվանդին՝ հաշվի առնելով հիվանդության ծանրությունը և ժամանման հերթականությունը: Ամենաբարձր ծանրություն ունեցող հիվանդը պետք է սպասարկվի առաջինը, իսկ հավասար ծանրության դեպքում՝ ավելի վաղ ժամանած հիվանդը: Սպասարկված հիվանդը չպետք է կրկին սպասարկվի:

Տեսական մաս. Մշակել ծրագիր, որը կօգտագործի `Patient` դասը՝ հիվանդանոցում հիվանդների հերթի առաջնահերթ կառավարումն իրականացնելու համար: Տվյալների ստուգման համար օգտագործել `while` ցիկլեր և `if` պայմաններ: Հիմնական գործողությունները կատարել դասի մեթոդների միջոցով:

- Սահմանում ենք `Patient` դասը՝ անհրաժեշտ դաշտերով:
- Դաշտերը հայտարարում ենք `private`, իսկ կոնստրուկտորն ու մեթոդները՝ `public`:
- Կոնստրուկտորի միջոցով սահմանում և ստուգում ենք հիվանդի տվյալները: Ծանրության աստիճանը պետք է լինի 1-ից 5 միջակայքում:
- Մուտքագրում և ստուգում ենք 15 հիվանդի տվյալները:
- `nextPatient()` մեթոդով ընտրում ենք չսպասարկված ամենաբարձր ծանրության հիվանդին: Հավասար ծանրության դեպքում ընտրում ենք ավելի վաղ ժամանած հիվանդին:
- `serve()` մեթոդով նշում ենք հիվանդին սպասարկված՝ կանխելով կրկնակի սպասարկումը:
- Հաշվում ենք յուրաքանչյուր ծանրության խմբում դեռ սպասող հիվանդների քանակը:
- Մուտքագրում ենք ընթացիկ ժամանակը՝ րոպեներով, և հաշվում յուրաքանչյուր սպասող հիվանդի սպասման ժամանակը:
- Գտնում ենք առավելագույն սպասման ժամանակը:
- Կազմում և արտածում ենք հիվանդների սպասարկման հաջորդականությունը:

Առաջադրանք. Ստեղծել C++ ծրագիր, որը կօգտագործի `Patient` դասը և 15 հիվանդի օբյեկտներից բաղկացած զանգված: Ծրագիրը պետք է ստուգի մուտքագրված տվյալների ճշտությունը, ընտրի հաջորդ հիվանդին՝ ըստ ծանրության աստիճանի և ժամանման ժամանակի, նշի սպասարկված հիվանդներին, հաշվի յուրաքանչյուր ծանրության խմբում սպասողների քանակը, որոշի առավելագույն սպասման ժամանակը և արտածի սպասարկման հաջորդականությունը: Սխալ մուտքագրման դեպքում պետք է արտածել սխալի հաղորդագրություն և կրկին պահանջել մուտքագրել ճիշտ արժեք:

```
#include <iostream>
#include <string>
using namespace std;
const int SIZE = 15;
class Patient {
private:
   int id;
   string name;
   int severity;
   int arrivalMinute;
   bool served;
public:
   Patient() {
      id = 0;
      name = "";
      severity = 1;
      arrivalMinute = 0;
      served = false;
   }
   Patient(int id, string name, int severity,
         int arrivalMinute, bool served) {
      this->id = id;
      this->name = name;
      this->severity = (severity >= 1 && severity <= 5)
                     ? severity
                     : 1;
      this->arrivalMinute = arrivalMinute;
      this->served = served;
   }
   // Ամբողջ թվի մուտքագրման ստուգում
   static int readInt(const string &prompt, int minValue,
                  int maxValue = 2147483647) {
      string input;
      while (true) {
         cout << prompt;
         if (!(cin >> input)) {
            cout << "Մուտքագրման սխալ։\n";
            cin.clear();
```

```
            cin.ignore(10000, '\n');
            continue;
        }
        bool valid = !input.empty();
        int value = 0;
        for (int i = 0; i < input.length(); i++) {
            if (input[i] < '0' || input[i] > '9') {
                valid = false;
                break;
            }
            int digit = input[i] - '0';
            if (value > (maxValue - digit) / 10) {
                valid = false;
                break;
            }
            value = value * 10 + digit;
        }
        if (valid && value >= minValue && value <= maxValue) {
            return value;
        }
        cout << "Սխալ մուտք։ Կրկին փորձեք։\n";
    }
}
// Հիվանդի տվյալների մուտքագրում
void input() {
    id = readInt("Մուտքագրեք հիվանդի ID-ն։ ", 1);
    cout << "Մուտքագրեք հիվանդի անունը։ ";
    cin >> name;
    while (name.empty()) {
        cout << "Անունը չի կարող դատարկ լինել։ ";
        cin >> name;
    }
    severity = readInt(
        "Մուտքագրեք ծանրության աստիճանը (1-5)։ ", 1, 5
    );
    arrivalMinute = readInt(
        "Մուտքագրեք ժամանման ժամանակը՝ րոպեներով։ ", 0
    );
    served = false;
}
int getId() const {
    return id;
}
int getSeverity() const {
    return severity;
}
int getArrivalMinute() const {
```

```
    return arrivalMinute;
  }
  bool isServed() const {
    return served;
  }
  // Արտածել տվյալները
  void print() const {
    cout << "ID: " << id
         << ", Անուն: " << name
         << ", Ծանրություն: " << severity
         << ", Ժամանում: " << arrivalMinute
         << " րոպե"
         << ", Կարգավիճակ: "
         << (served ? "Սպասարկված" : "Սպասում է")
         << '\n';
  }
  // Սպասարկել հիվանդին
  bool serve() {
    if (served) {
      return false;
    }
    served = true;
    return true;
  }
  // Հաշվել սպասման ժամանակը
  int waitingTime(int currentMinute) const {
    return currentMinute - arrivalMinute;
  }
};
// Գտնել հաջորդ սպասարկվող հիվանդին
int nextPatient(Patient patients[], int size) {
  int index = -1;
  for (int i = 0; i < size; i++) {
    if (!patients[i].isServed()) {
      if (index == -1 ||
        patients[i].getSeverity() >
        patients[index].getSeverity() ||
        (patients[i].getSeverity() ==
         patients[index].getSeverity() &&
         patients[i].getArrivalMinute() <
         patients[index].getArrivalMinute())) {
        index = i;
      }
    }
  }
  return index;
```

```
}
// Հաշվել սպասողների քանակն ըստ ծանրության
void countWaitingBySeverity(Patient patients[], int size) {
   cout << "\nՍպասողների քանակն ըստ ծանրության խմբերի՝\n";
   for (int severity = 1; severity <= 5; severity++) {
      int count = 0;
      for (int i = 0; i < size; i++) {
         if (!patients[i].isServed() &&
            patients[i].getSeverity() == severity) {
            count++;
         }
      }
      cout << "Ծանրություն " << severity
           << ": " << count << " հիվանդ\n";
   }
}
// Գտնել առավելագույն սպասման ժամանակը
void findMaximumWaitingTime(
   Patient patients[], int size, int currentMinute
) {
   int maxWaiting = 0;
   bool found = false;
   for (int i = 0; i < size; i++) {
      if (!patients[i].isServed()) {
         int waiting = patients[i].waitingTime(currentMinute);

         if (!found || waiting > maxWaiting) {
            maxWaiting = waiting;
            found = true;
         }
      }
   }
   cout << "\nԱռավելագույն սպասման ժամանակը: ";
   if (found) {
      cout << maxWaiting << " րոպե\n";
   } else {
      cout << "Սպասող հիվանդներ չկան։\n";
   }
}
// Կազմել սպասարկման հաջորդականությունը
void serviceSequence(Patient patients[], int size) {
   cout << "\nՍպասարկման հաջորդականությունը՝\n";
   int count = 0;
   while (true) {
      int index = nextPatient(patients, size);
```

```
        if (index == -1) {
            break;
        }
        count++;
        patients[index].serve();
        cout << count << ". ";
        patients[index].print();
    }
    if (count == 0) {
        cout << "Բոլոր հիվանդներն արդեն սպասարկված են։\n";
    }
}
int main() {
    Patient patients[SIZE];
    cout << "ՀԻՎԱՆԴԱՆՈՑԻ ՀԵՐԹԻ ԿԱՌԱՎԱՐՈՒՄ\n";
    // Մուտքագրել հիվանդների տվյալները
    for (int i = 0; i < SIZE; i++) {
        cout << "\n--- " << i + 1
             << "-րդ հիվանդ ---\n";
        while (true) {
            patients[i].input();
            bool duplicate = false;
            for (int j = 0; j < i; j++) {
                if (patients[i].getId() == patients[j].getId()) {
                    duplicate = true;
                    break;
                }
            }
            if (!duplicate) {
                break;
            }
            cout << "Այս ID-ն արդեն օգտագործված է։ "
                 << "Կրկին մուտքագրեք տվյալները։\n";
        }
    }
    // Ստուգել ընթացիկ ժամանակը
    int currentMinute;
    while (true) {
        currentMinute = Patient::readInt(
            "\nՄուտքագրեք ընթացիկ ժամանակը՝ րոպեներով։ ", 0
        );
        bool validTime = true;
        for (int i = 0; i < SIZE; i++) {
            if (patients[i].getArrivalMinute() > currentMinute) {
                validTime = false;
                break;
            }
```

```
        }
        if (validTime) {
            break;
        }
        cout << "Ընթացիկ ժամանակը չի կարող ավելի փոքր լինել "
             << "հիվանդի ժամանման ժամանակից։ Կրկին փորձեք։\n";
    }
    // Արտածել բոլոր հիվանդներին
    cout << "\nԲոլոր հիվանդների տվյալները՝\n";
    for (int i = 0; i < SIZE; i++) {
        patients[i].print();
    }
    // Հաշվել սպասողների քանակն ըստ ծանրության
    countWaitingBySeverity(patients, SIZE);
    // Հաշվել առավելագույն սպասման ժամանակը
    findMaximumWaitingTime(patients, SIZE, currentMinute);
    // Կազմել սպասարկման հաջորդականությունը
    serviceSequence(patients, SIZE);
    cout << "\nԾրագիրն ավարտվեց։\n";
    return 0;
}
```